



Build with AI. Think for yourself.

Intelligent AI Dependency Regulation System

NiyantraAI measures whether a developer's AI usage is turning into genuine understanding. It does not count AI requests, and it never recommends using AI less. It measures the part where your own reasoning drops out of the loop.

SUBMISSION

[iQOO Hackathon](#)

LIVE DEMO

theleverager1705.github.io/niyantraai-iqoo-hackathon

SOURCE

github.com/theLeverager1705/niyantraai-iqoo-hackathon

DOCUMENT

Technical documentation, v1.0

All scores described in this document are a prototype heuristic, not a validated psychometric instrument.

01 OVERVIEW

What NiyantraAI is

Modern AI assistants can generate an entire application, debug it, design its architecture and document it. The code works. The demo passes. And then something breaks in production, and nobody on the team has a model of how it fits together.

This is a new failure mode, and it is invisible to every existing measure of developer productivity. Shipping velocity goes up. Understanding quietly goes down. Nothing in the toolchain notices.

PRODUCT THESIS

AI usage is not the problem. Unconscious dependence is. NiyantraAI is built to optimise, never to restrict. Dependency is scored against a healthy band rather than a slope, and no recommendation the product can generate will ever tell a developer to use AI less.

What it measures

Seven days of AI-assisted development activity are reduced to five metrics and one combined score. Every number is computed by a running engine from countable behaviour, and every number can be traced back to the raw signals that produced it inside the product itself.

68.6% AI Dependency	73.4% Independent Thinking	80.5% Code Understanding	85.5% Prompt Quality	+16 Learning Momentum	73 Balance / 100
------------------------	-------------------------------	-----------------------------	-------------------------	--------------------------	---------------------

Figures above are the live values the demo profile produces. They are not written into the interface; they are the output of the analytics engine running over 190 seeded interaction records, and `npm run calibrate` prints the same numbers from the command line.

The product in one screen list

SCREEN	THE QUESTION IT ANSWERS
Dashboard	Is AI amplifying me or replacing me, right now?
AI Analytics	What raw behaviour produced that verdict?
Prompt Intelligence	How do I ask, not how often?
Project Understanding	Do I understand the thing I shipped?
Adaptive Assessment	Where exactly is the edge of what I know?
Your AI Coach	What is the single highest-impact change this week?
Weekly Report	What improved, what needs attention, what next?
Achievements	Which behavioural benchmarks am I currently meeting?
Integrations	Where could this data honestly come from?
Privacy and Settings	What is being measured, and can I stop it?

Plus a landing page, a four-step onboarding flow and an eight-question baseline assessment: thirteen screens in total.

02 THE DIFFERENTIATOR

Do you understand what you built?

This is the feature the product exists for. A developer should not receive a high skill score simply because they produced a working application with AI. Working software is evidence that the code runs. It is not evidence that anyone understands it.

When a project is connected, NiyanttraAI generates an adaptive assessment about that specific system. The questions are not trivia; they are the questions a senior engineer would ask in a design review.

EXAMPLE QUESTIONS

- What happens between the moment the user submits the login form and the moment authentication succeeds?
- Why did you choose JWT instead of server-side sessions? Give the trade-off you accepted, not just the benefit.
- If the database becomes unavailable, which part of your application fails first?
- Explain this function in your own words. What is it protecting against?

How answers are evaluated

Free text is scored on **concept coverage and causal reasoning**, never on similarity to a model answer. Each question declares the concepts a complete answer touches, with acceptable phrasings, so a developer who explains the mechanism in their own words scores as well as one who happens to use the textbook vocabulary.

```
score = clamp( coverage * 68 + depth * 24 + detail_bonus - misconception_penalty )
```

coverage	share of the question's expected concepts the answer touches
depth	causal language (because, so that, otherwise, when) + answer length
detail	+8 once the answer passes twelve words
penalty	-12 per misconception the question declares and the answer contains

Partial answers score partially, and the feedback names precisely what was left out. Misconceptions are detected and corrected by name rather than silently marked wrong.

Verified behaviour

Both directions of the loop were exercised end to end on the deployed build:

ANSWER GIVEN	RESULT	ENGINE RESPONSE
"The middleware runs first because every route needs the body parsed, the user authenticated and the payload validated... each middleware either ends the response or calls next()."	96 / 100	Difficulty stepped Intermediate to Advanced. Next question served was the JWT architecture question.
"JWT is more secure than sessions so I used it."	0 / 100	Misconception named explicitly. Difficulty stepped Advanced back to Intermediate. Missing concepts listed: stateless, revoke, expiry.

SCORES MOVE

Completing that five-question check dropped the demo profile's project understanding from 83% to 77%, and its Backend area from 79% to 70% with the note rewritten to "you can describe what happens here; the why is still partly borrowed". The dashboard's Code Understanding metric and AI-Human Balance moved with it. The assessment is wired into the scoring model, not bolted beside it.

03 THE SCORING MODEL

Transparent by construction

Every score is a weighted blend of observable behaviour. The weights are published inside the product on the dashboard and on the analytics screen, because a number a developer cannot interrogate is a number they will not trust.

HONESTY CONSTRAINT

These weights are chosen to be **explainable**. They are a prototype heuristic and are not a validated psychometric instrument. The product states this wherever a score appears, rather than in a footnote.

AI Dependency

How much of the developer's problem-solving was handed over rather than reasoned through first. Five countable inputs, each normalised to 0-1.

WEIGHT	SIGNAL	DEFINITION
28%	Handover intensity	Mean assistance level across requests, where 5 means the assistant produced the whole artefact
22%	Substitution share	Requests asking for code generation or a complete solution
26%	Not attempted first	Problems opened in an assistant without a prior attempt
10%	Unexamined acceptance	Accepted suggestions kept without a single edit
14%	AI code authorship	Share of the week's lines that came from the assistant

The healthy band

Dependency is deliberately **not** scored as linearly bad. A developer using AI heavily for leverage is doing their job. The risk sits above roughly 55%, where output starts outpacing understanding, and mildly below 20%, where available leverage is going unused.

```
dependency_health(d):  
    if d > 55: 100 - (d - 55) * 3.0    # steep: the runaway end  
    if d < 20: 100 - (20 - d) * 1.4    # gentle: unused leverage  
    else:     100                     # the healthy band
```

This single design decision is what makes the product's stated principle true in the mathematics rather than only in the copy. The demo profile sits at 69% dependency and is still rated **Healthy** overall, because the rest of its behaviour compensates.

Independent Thinking

Scored against healthy *targets* rather than perfection, because "100% of problems attempted alone first" is neither realistic nor desirable. Each input is capped at its benchmark, so exceeding a target cannot mask a deficit elsewhere.

WEIGHT	BEHAVIOUR	HEALTHY TARGET
30%	Attempted the problem before asking	50%
18%	Edited accepted suggestions before keeping them	58%
18%	Asked a follow-up, showing the answer was read	58%
18%	Requests seeking understanding, not output	40%
16%	Lines written by hand rather than generated	36%

Code Understanding

Deliberately a blend, because either half alone is gameable. Behaviour alone can be performed; an assessment alone is a single snapshot on a single day.

```
code_understanding = 0.55 * assessment_evidence + 0.45 * observed_behaviour

assessment_evidence  code reading, debugging and conceptual scores, weighted
                      70% toward project-specific answers over the older baseline
observed_behaviour   edit rate, follow-up rate and explanation share, each
                      measured against its healthy benchmark
```

The consequence is the point: **a high Code Understanding score cannot be earned by shipping**. Half of it is only available by answering questions about the system.

AI-Human Balance Score

The single headline number, combining all four inputs.

```
balance = 0.34 * human_skill
          + 0.30 * dependency_health      # the band function, not raw dependency
          + 0.20 * prompt_quality
          + 0.16 * independent_thinking

human_skill = 0.45 * code_understanding
              + 0.30 * independent_thinking
              + 0.25 * assessment_overall
```

Bands: 70 and above is Healthy, 55-69 Moderate, 42-54 Elevated, below 42 High risk. The dependency metric carries its own separate band, so a profile can legitimately read "Moderate dependency, Healthy balance" at the same time.

Learning Momentum

Week-over-week change in a capability index that is intentionally anchored by the slower-moving assessment baseline, so a single strong week cannot spike it and a single quiet week cannot crater it.

```
capability = 0.30 * independent_thinking + 0.30 * code_understanding
             + 0.15 * prompt_quality      + 0.25 * assessment_baseline

momentum   = capability(this week) - capability(previous week)
```

Calibration against the brief

The product brief specified illustrative target figures. Rather than hardcode them, the seeded dataset and model were tuned until the engine genuinely produced them:

METRIC	BRIEF	COMPUTED	SOURCE
AI Dependency	68%	68.6%	engine output, -10.4 pts week over week
Independent Thinking	74%	73.4%	engine output
Code Understanding	81%	80.5%	engine output
Prompt Quality	87%	85.5%	mean of 15 live-scored prompts
Learning Momentum	+14%	+16 pts	engine output
AI-Human Balance	72 / 100	73 / 100	rated Healthy
Strong prompt example	94	94	rubric output, exact match
Risky prompt example	41	41	rubric output, exact match

04 PROMPT INTELLIGENCE

How you ask, not how often

Counting AI requests measures nothing useful. "Here is my approach, identify the flaws without giving me the solution" and "write the complete implementation" are both one request, and they produce entirely different developers.

Every captured prompt is scored against a published rubric of weighted signals. The Prompt Intelligence screen includes a live lab: paste any prompt and watch the same rubric apply itself signal by signal.

The rubric

+	SIGNAL THAT KEEPS YOU IN THE LOOP	–	SIGNAL THAT HANDS IT OVER
16	Shows your own attempt	8	Hands the problem over
15	Asks for critique, not code	6	Unbounded scope
12	Withholds the solution deliberately	5	No context provided
10	Concrete context supplied	4	Requests a complete implementation
9	Probes why / trade-offs	3	Too short to carry intent
7	Scoped to one problem		
6	Enough detail to be answerable		

```
score = clamp( 53 + positive_weights * 0.62 - negative_weights )
```

The two prompts, scored live

STRONG — SCORED 94

"Here is my approach to implementing binary search over the ranked resume matches. Identify flaws without giving me the solution."

Shows your own attempt + asks for critique, not code + withholds the solution deliberately + concrete context + scoped to one problem. You remain the author of the solution; the assistant becomes a reviewer.

RISKY — SCORED 41

"Write the complete implementation for this problem."

Requests a complete implementation, no context, too short to carry intent. You receive an artefact instead of building a model of it.

The insight this screen surfaces

THE PATTERN WORTH NOTICING

The demo profile scores **85.5 on prompt quality while sitting at 68.6% dependency**. That is not a contradiction, and the product says so explicitly: this developer is very good at asking, and they are asking for finished work. The skill is real. It is pointed one level too high.

The recommendation that follows is therefore not "write better prompts". It is "**keep the same prompt structure, change the verb**": *generate X* becomes *here is my X, where does it break?*

05 THE ADAPTIVE ENGINE

Finding the edge of what you know

A fixed quiz measures whether someone saw the material. An adaptive one finds the boundary of what they actually understand. The engine is deliberately legible: difficulty moves one step at a time on published thresholds, and the interface shows the move as it happens.

RULE	BEHAVIOUR
Score 75 or above	Difficulty steps up one level. The concept is treated as held, not guessed.
Score below 45	Difficulty steps down, and the next question targets the concept underneath the one missed.
Coverage before depth	Dimensions not yet assessed are favoured, so one strong area cannot carry the score.
Stack breaks ties	At equal fit, questions tagged with the developer's own technologies win.
Format varies	Repeating the previous question format is penalised, so it does not read as a quiz.

Difficulty ladder: **Foundational to Intermediate to Advanced**. The starting rung comes from the level declared during onboarding. Question formats span multiple choice, debugging, code comprehension, architecture and short answer.

Question bank

SET	COUNT	PURPOSE
Baseline	8	A fixed yardstick every developer sees in the same order, captured before heavy AI assistance
Project	10	Architecture, backend, database, AI integration and frontend questions about the shipped system
Advanced	8	Concurrency, memory, cost, index trade-offs and reading unfamiliar code

Each free-text question declares its expected concepts with acceptable phrasings, the misconceptions that indicate a wrong mental model, a model answer revealed only after submission, and tags used for stack-relevant selection.

Baseline assessment

Eight questions taken during onboarding, before extensive AI assistance, establishing the reference point that every later measurement is relative to. It reports per-dimension results rather than a single pass mark:

62% Technical Understanding	58% Problem Solving	71% Code Reading	49% Architecture
---------------------------------------	-------------------------------	----------------------------	----------------------------

There is no pass mark, and the interface says so. A low baseline simply makes later growth measurable, and the weakest dimension becomes the coach's first target.

06 COACHING AND REPORTING

Evidence, never generic advice

Every coaching card is generated from the developer's own numbers. A card cannot exist unless the behaviour that triggers it is present in the data, and each one states the evidence that produced it. There is no library of generic tips.

A GENERATED CARD, VERBATIM FROM THE DEMO PROFILE**Attempt problems independently before requesting implementations**

Why. Attempting first is the single behaviour most correlated with being able to explain your own codebase later. It is also the cheapest to change.

Evidence from your week. You requested complete solutions 20 times this week, while independent attempts were recorded 27 times across 91 interactions (30% attempt-first, healthy target 50%).

Try this. Spend ten minutes designing the solution yourself. Then ask the assistant to critique your approach rather than generate it.

Cards are ranked **primary**, **supporting** and **reinforce**. The reinforce tier matters as much as the others: it names the habits already working, so a developer short on time knows which behaviour to protect rather than which to add.

The independent challenge

The primary card's action opens a working fifteen-minute challenge with a real timer, a brief, explicit rules and reflection prompts. Four challenges ship: trace a request end to end, design before you generate, debug without asking, and rebuild one generated function. Each produces an artefact the developer can then hand to an assistant *for critique*, which is the behaviour change the whole product is arguing for.

Weekly growth report

Assembled from the same signals, with week-over-week comparison computed per weekday rather than as a single aggregate. Sample output from the demo profile:

WHAT IMPROVED	WHAT NEEDS ATTENTION
AI dependency fell 10.4 points, from 79% to 69%.	Dependency is still 69%, above the 55% point where output starts outpacing understanding.
You reduced complete-solution prompts by 33% (30 to 20).	62% of requests asked for generation or a complete solution.
Independent thinking rose 24.4 points, driven by an attempt-first rate of 30% against 14% last week.	Architecture understanding remains the weakest measured dimension.

Gamification, kept professional

Eight achievements, all behavioural benchmarks with visible thresholds and progress: Independent Thinker, AI Strategist, Debugging Pro, Architecture Explorer, Prompt Engineer, The Explainer, Knows What They Shipped, Consistency. They are recomputed from the current week on every load, which means they can be lost again if the behaviour stops. Developer XP is derived from attempts, follow-ups, explanation requests, edits and assessments, and never from time spent in the application.

07 ARCHITECTURE

One seam, deliberately placed

The entire product is built around a single abstraction: the **AI Usage Provider**. Swap the provider and every layer above it keeps working unchanged. That seam is what makes the prototype a credible foundation rather than a demonstration.

```
interface AIUsageProvider {
  id, label, mode: 'simulated' | 'live'
  isAvailable(): boolean
  fetchInteractions(query: UsageQuery): Promise
  describe(): IntegrationStatus[]
}
```

The prototype registers one implementation, `SimulatedUsageProvider`, which reads a seeded dataset and applies the same query filtering a live provider would. Nothing downstream knows the difference.

Source layout

```
src/
  types/      Domain model: User, Project, AIInteraction, PromptAnalysis,
              Assessment, Question, SkillMetric, Recommendation, WeeklyReport
  data/       Taxonomy, seeded dataset, prompt bank, question bank, challenges
  lib/        Pure logic, zero React imports:
              analyticsEngine.ts    scoring model + published methodology
              promptAnalyzer.ts     the prompt rubric
              answerEvaluator.ts     concept-coverage evaluation
              assessmentEngine.ts    adaptive selection + difficulty ladder
              recommendations.ts     coach card generation
              weeklyReport.ts        report assembly
              achievements.ts        benchmarks + XP
  services/   Orchestration and the integration seam:
              providers/aiUsageProvider.ts  the interface + registry
              providers/simulatedProvider.ts prototype implementation
              mockAIAalyticsService.ts       pulls activity, runs lib/
              mockAssessmentService.ts       engine + evaluator
              mockGitHubService.ts           repository analysis
              llm/llmService.ts              LLM abstraction
              storage.ts                     local persistence
  hooks/      useAppState.tsx (store), useMediaQuery.ts
  components/ ui/ primitives, layout/, charts/, dashboard/,
              assessment/, coach/
  pages/      One file per screen
```

The separation is enforced rather than aspirational: `lib/` contains the entire scoring and assessment model and imports no React at all, which is why the same code runs under Node for the calibration script.

Tech stack

LAYER	CHOICE	WHY
Framework	React 18.3 + TypeScript 5.6 (strict)	Strict mode on, zero type errors in CI
Build	Vite 5.4	Relative base and code splitting into three chunks
Styling	Tailwind CSS 3.4	Custom dark-first token set, no component library
Charts	Recharts 2.13	Interactive, accessible summaries on every figure
Routing	React Router 6 (HashRouter)	So the build also runs from disk and any sub-path
Icons	lucide-react	Single consistent icon family

08 DATA MODEL

What the product reasons about

The domain model is the contract between simulated and real data. It is written once, in `src/types/index.ts`, and every layer depends on it rather than on the provider.

```
interface AIInteraction {
  id, timestamp, tool
  category
  chatgpt | claude | gemini | copilot | cursor
  learning | debugging | code-generation |
  architecture | explanation |
  complete-solution | optimization

  promptText?, promptLength
  assistanceLevel: 1..5
  1 = you reasoned and AI reviewed
  5 = AI produced the whole artefact
  independentlyAttempted: boolean
  tried before opening the assistant
  generatedCode, manualCode
  lines, by origin
  userAccepted, userModified
  was it kept, was it read
  followUpAsked: boolean
  evidence the answer was engaged with
  sessionMinutes
}
```

`assistanceLevel`, `independentlyAttempted` and `userModified` carry most of the product's signal. They are also the three fields only an editor integration can observe directly, which is why the roadmap leads with the extension.

Principal types

TYPE	ROLE
User, Project	Identity, declared level, area, tools, stack, repository
AllInteraction	One request to an assistant and everything observable about it
PromptAnalysis, PromptSignal	Rubric output: score, verdict, and each signal with its weight
Assessment, Question, AssessmentResponse	Sessions, the bank, and per-answer scores with feedback
SkillMetric, BalanceScore	Metric with delta and direction; the combined score with contributions
ProjectUnderstanding, UnderstandingArea	Per-area understanding profile with rewritten notes
Recommendation, WeeklyReport	Coach cards with evidence; assembled weekly output
RepoAnalysis, RepoModule	Repository structure, languages, complexity, AI-authored share
PrivacySettings, IntegrationStatus	Consent state and the honest capability of each source

The seeded dataset

190 interaction records across fourteen days: 91 in the current window, 99 in the comparison window. Generated from a fixed seed with a `mulberry32` PRNG, so every machine and every reload produces byte-identical data.

Behavioural rates are assigned by **exact quota** rather than independent coin flips. With roughly ninety records a week, Bernoulli sampling noise was large enough to invert the week-over-week story the demo needs to tell. The generator ranks interactions by a realistic propensity, then applies the quota, so the pattern still reads like a person while the aggregate is guaranteed.

09 REAL AND SIMULATED

Stated precisely, not disclaimed

Being exact about this is a product requirement. A judge should be able to tell instantly which parts are engineering and which parts are scaffolding.

Running code

- The analytics engine, prompt rubric, answer evaluator and adaptive assessment engine all execute on every load. No displayed number is hardcoded.
- The dashboard figures are computed from 190 seeded records by the same engine a live provider would feed. `npm run calibrate` prints them from the command line.
- Assessment answers are evaluated live, and results write back into the understanding profile and the dashboard metrics.
- Repository analysis, privacy toggles, data export and delete, and the challenge timer are all functional. There are no dead buttons.

Simulated, and labelled in the interface

- **AI activity feed** (`data/demoDataset.ts`) - seeded generation, marked "Simulated AI Activity" on every screen that consumes it.
- **Repository analysis** (`services/mockGitHubService.ts`) - deterministic per repository URL, shaped exactly like the real GitHub API response, and badged "Simulated in this prototype".
- **Answer evaluation** (`services/llm/llmService.ts`) - defaults to an on-device concept-coverage rubric. Setting `VITE_LLM_API_KEY` routes to any OpenAI-compatible endpoint instead, falling back silently on any error so a demo cannot be broken by the network.

WHAT THIS PROJECT WILL NOT CLAIM

There is **no API that grants a third party access to a private ChatGPT or Claude history**. Any real integration is either an editor extension running locally with the developer's consent, or a conversation export they choose to upload. The Integrations screen states this plainly for every provider rather than implying access the product could not obtain.

Integration roadmap, honestly graded

SOURCE	STATUS	WHAT IT CAN ACTUALLY PROVIDE
VS Code / Cursor extension	Needs extension	Acceptance, modification, manual-vs-generated line counts. The only source that can measure whether generated code was read.
GitHub	Available	Repository analysis, commit cadence, co-authorship trailers, project-specific question generation
GitHub Copilot	Planned	Acceptance rate and suggestion volume, but only at organisation level via the metrics API
ChatGPT / Claude / Gemini	Planned	Prompt text, category and timestamps, from an export the user uploads themselves

10 PRIVACY

A design constraint, not a policy page

A product that analyses developer behaviour has to answer the privacy question in its architecture, not in a document. Everything NiyanttraAI knows lives in the developer's own browser. There is no server, no account and no third party.

CONTROL	EFFECT WHEN DISABLED
AI activity tracking	Interactions with assistants are no longer recorded
GitHub access	Repository structure and commit metadata are not read
Analytics collection	Measurement genuinely stops; the dashboard empties and says so
Prompt text retention	Scores are still computed, but no prompt examples are kept
Anonymised benchmarks	Off by default, and never includes prompt text

Full JSON export and delete are both implemented. Turning analytics off stops measurement rather than hiding it, and the empty state explains that no history was kept while it was disabled. Every storage access is guarded, so a private-mode browser degrades to an in-memory session instead of crashing.

Running it

```
npm install
npm run dev      # http://localhost:5173
```

Requires Node 18 or newer. No API keys, no accounts, no database, no network calls.

COMMAND	DOES
npm run dev	Development server with hot reload
npm run build	Typecheck, then production build to dist/
npm run preview	Serve the production build locally
npm run typecheck	TypeScript only
npm run calibrate	Print every number the engine produces for the demo dataset

The build uses a relative base and a HashRouter, so `dist/index.html` also works opened directly from disk or served from any sub-path. Deployment is automated: pushing to `main` typechecks, builds and publishes to GitHub Pages.

Quality bar met

- Zero TypeScript errors under strict mode; zero console errors in the browser.
- Loading, empty, error and success states on every analytics-backed screen, routed through a single shared gate so they are identical everywhere.
- Responsive from 375px, with a purpose-built mobile layout and bottom tab bar rather than a compressed desktop page.
- Semantic HTML, keyboard navigation, a skip link, visible focus rings, accessible chart summaries, and reduced-motion support.
- An error boundary turns any render failure into a recoverable screen rather than a blank page.

11 DEMONSTRATION

A three to five minute walkthrough

Demo Mode loads a pre-populated profile instantly: Alex, an intermediate web developer building an AI Resume Analyzer with ChatGPT, Claude and GitHub Copilot, with fourteen days of history behind them. No signup, no configuration.

	SCREEN	WHAT TO SAY AND DO
1	Landing	Open the live URL and press Explore Dashboard .
2	Dashboard	Balance 73/100, rated Healthy. Point at the band gauge : dependency is 69% and the product still says Healthy. That is the thesis, visible in the mathematics.
3	Prompt Intelligence	The 94 and the 41, side by side, each broken down by signal. Then type a prompt into the live lab and watch the score move.
4	Project Understanding	Press Start knowledge check . Answer the middleware question properly: score around 96, difficulty steps Intermediate to Advanced, and the JWT question appears.
5	The reversal	Answer the JWT question with "JWT is more secure than sessions". Score 0, the misconception is named, and difficulty steps back down. This is the strongest forty-five seconds of the demo.
6	Back to Dashboard	The scores have moved. Understanding fell 83 to 77, Backend 79 to 70, and the balance followed.
7	AI Coach	The recommendation, its evidence in real counts, and a working fifteen-minute independent challenge.
8	Weekly Report	What improved, what needs attention, what to do next.

What remains for a production version

The prototype is honest about its boundary. These are the next steps in priority order:

- **Ship the editor extension.** VS Code and Cursor are the only sources that can observe whether generated code was actually read. Every acceptance and modification metric depends on it, and it is the difference between inference and measurement.
- **Real GitHub analysis.** Replace the deterministic mock with live API calls and generate questions from the project's actual AST rather than from declared stack tags.
- **Model-based evaluation by default**, with the current concept-coverage rubric retained as the offline fallback it already is.
- **Server-side history**, so trends run longer than two weeks. The momentum metric is designed for a longer window than the prototype can currently give it.
- **Validate the weights against outcomes.** Until that work is done, the product is right to call its scores a heuristic, and it does so on every screen.

Build with AI. Understand what you build.

NiyanttraAI · iQOO Hackathon · github.com/theLeverager1705/niyantraai-iqoo-hackathon